```
# 06_gradcam_and_errors.py
# Grad-CAM heatmaps (RGB-only ResNet vs NIR+NDVI ResNet) and error analysis by weather
# condition. Needs the GPU runtime for speed but also works on CPU. Run after 05.
```

```
# %% [1] Imports and settings
import os, json
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

ROOT = os.environ.get("AMAZON_ROOT", "/content/drive/MyDrive/amazon_project")
try:
    from google.colab import drive
    drive.mount("/content/drive")
except ImportError:
    pass

CACHE, RUNS, FIG = f"{ROOT}/cache", f"{ROOT}/runs", f"{ROOT}/figs"
os.makedirs(FIG, exist_ok=True)
DEV = "cuda" if torch.cuda.is_available() else "cpu"
LABELS = json.load(open(f"{CACHE}/labels.json"))
NL = len(LABELS)
X = np.load(f"{CACHE}/X.npy", mmap_mode="r")
st = np.load(f"{CACHE}/stats.npz")
MEAN, STD = st["mean"], st["std"]
CHANS = {"C_resnet_rgb": [0, 1, 2], "D_resnet_nir_ndvi": [0, 1, 2, 3, 4]}
TARGETS = [l for l in ["selective_logging", "slash_burn", "artisinal_mine", "agriculture"]
           if l in LABELS]
```

```
# %% [2] Load both ResNet models and their predictions
def build_resnet(c_in):
    m = torchvision.models.resnet34(weights=None)
    m.conv1 = nn.Conv2d(c_in, 64, 7, 2, 3, bias=False)
    m.fc = nn.Linear(m.fc.in_features, NL)
    return m


models, data = {}, {}
for n, ch in CHANS.items():
    m = build_resnet(len(ch))
    m.load_state_dict(torch.load(f"{RUNS}/{n}.pt", map_location="cpu"))
    models[n] = m.to(DEV).eval()
    d = np.load(f"{RUNS}/{n}_probs.npz")
    thr = np.array([json.load(open(f"{RUNS}/thresholds_{n}.json"))[l] for l in LABELS])
    data[n] = dict(pred=d["test_probs"] >= thr[None, :], y=d["test_y"], idx=d["test_idx"])
assert (data["C_resnet_rgb"]["idx"] == data["D_resnet_nir_ndvi"]["idx"]).all()
TEST_IDX = data["C_resnet_rgb"]["idx"]
Y = data["C_resnet_rgb"]["y"]
```

```
# %% [3] Grad-CAM implementation
def gradcam(model, x, label_j):
    acts, grads = [], []
    h = model.layer4.register_forward_hook(
        lambda mod, i, o: (acts.append(o), o.register_hook(lambda g: grads.append(g))))
    model.zero_grad()
    out = model(x)
    out[0, label_j].backward()
    h.remove()
```

```
    w = grads[0].mean((2, 3), keepdim=True)
    cam = F.relu((w * acts[0]).sum(1, keepdim=True))
    cam = F.interpolate(cam, size=x.shape[-2:], mode="bilinear", align_corners=False)[0, 0]
    cam = cam - cam.min()
    return (cam / (cam.max() + 1e-8)).detach().cpu().numpy()


def prep(gi, n):
    ch = CHANS[n]
    x = torch.from_numpy(np.asarray(X[gi])[ch].astype(np.float32))[None].to(DEV)
    mean = torch.tensor(MEAN[ch], device=DEV).view(1, -1, 1, 1).float()
    std = torch.tensor(STD[ch], device=DEV).view(1, -1, 1, 1).float()
    return (x - mean) / std


def rgb_view(gi):
    return np.clip(np.transpose(np.asarray(X[gi])[:3].astype(np.float32), (1, 2, 0)) ** 0.8, 0, 1)


# %% [4] Heatmaps for tiles where NDVI "fixes" a miss, and where both models miss
C, D = data["C_resnet_rgb"], data["D_resnet_nir_ndvi"]
for lab in TARGETS:
    j = LABELS.index(lab)
    truth = Y[:, j] == 1
    fixed = np.where(truth & ~C["pred"][:, j] & D["pred"][:, j])[0][:3]
    both = np.where(truth & ~C["pred"][:, j] & ~D["pred"][:, j])[0][:2]
    picks = [("D fixes C", i) for i in fixed] + [("both miss", i) for i in both]
    if not picks:
        print("no example tiles for", lab)
        continue
    fig, axes = plt.subplots(len(picks), 3, figsize=(8, 2.7 * len(picks)), squeeze=False)
    for r, (kind, i) in enumerate(picks):
        gi = int(TEST_IDX[i])
        base = rgb_view(gi)
        axes[r][0].imshow(base)
        axes[r][0].set_title(f"{kind} ({lab})", fontsize=8)
        for c, n in enumerate(["C_resnet_rgb", "D_resnet_nir_ndvi"], start=1):
            cam = gradcam(models[n], prep(gi, n), j)
            axes[r][c].imshow(base)
            axes[r][c].imshow(cam, cmap="jet", alpha=0.45)
            axes[r][c].set_title("RGB-only model" if c == 1 else "NIR+NDVI model", fontsize=8)
        for a in axes[r]:
            a.axis("off")
    plt.tight_layout()
    plt.savefig(f"{FIG}/fig_gradcam_{lab}.png", dpi=200)
    plt.close()
    print(lab, "examples:", len(picks))

# %% [5] Error analysis: where do mistakes come from? (weather condition per tile)
weather = [w for w in ["clear", "partly_cloudy", "haze", "cloudy"] if w in LABELS]
rows = []
for n in CHANS:
    wrong = (data[n]["pred"].astype(int) != Y).mean(1)           # share of 17 labels wrong
    for w in weather:
        m = Y[:, LABELS.index(w)] == 1
        rows.append(dict(model=n, weather=w, tiles=int(m.sum()),
                         mean_label_error=float(wrong[m].mean())))
err = pd.DataFrame(rows)
err.to_csv(f"{FIG}/error_by_weather.csv", index=False)
print(err.round(4).to_string())
pv = err.pivot(index="weather", columns="model", values="mean_label_error").loc[weather]
pv.plot.bar(figsize=(7, 4), color=["#9DB5A6", "#FF8A3D"])
plt.ylabel("share of the 17 labels predicted wrong")
plt.title("Error by weather condition (test set)")
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig(f"{FIG}/fig_error_by_weather.png", dpi=200)
```

```
plt.close()
print("Done. Use fig_gradcam_*.png and fig_error_by_weather.png on slide 15.")
```